PROJECT TITLE

SpendWise

A Secure, Full-Stack Intelligent Personal Expense Management, Budget Optimization and Financial Health Analytics Platform

A Project Work Synopsis

Submitted in the partial fulfilment for the award of the degree of

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING

STUDENT NAME & UID

[STUDENT_NAME_1] — [UID_1]
[STUDENT_NAME_2] — [UID_2]

Under the Supervision of:

[Faculty Guide Name]
[Designation, Department of Computer Science & Engineering]

CU CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

CHANDIGARH UNIVERSITY, GHARUAN, MOHALI - 140413, PUNJAB
October, 2026

Table of Contents

1. Introduction **1**

1.1 Background 1

1.2 Project Overview 1

1.3 Motivation and Scope 2

2. Problem Statement **4**

2.1 Core Problem 4

2.2 Technical Challenges 4

2.3 Constraints and Assumptions 5

3. Objectives **6**

3.1 Main Objective 6

3.2 Specific Objectives 6

4. Proposed Solution **7**

4.1 Limitations of Existing Systems 7

4.2 Overview of the Proposed Solution 7

4.3 System Architecture 9

4.4 Methodology 10

4.5 Algorithmic and Relational Design 11

4.6 Financial Health Scoring Rubric and Deterministic Generation 12

4.7 API Call Handling, Middleware and Error Logic 13

4.8 State Management and Client Synchronization 14

5. Modules / Key Features **15**

5.1 Module Structure 15

5.2 User Authentication and Session Management 16

5.3 Transaction Ledger Ingestion and Batch Management 16

5.4 Dynamic Budget Allocation and Variance Tracking 16

5.5 Financial Health and Predictive Run-Rate Engine 16

5.6 Deep Analytical Reporting Across Time Horizons 16

5.7 Payment Mode and Day-of-the-Week Habit Analysis 16

5.8 Interactive Interface and Visualisation 17

5.9 Printable Financial Statement and CSV Export 17

6. Technology Stack **18**

6.1 Software and Libraries 18

6.2 Hardware Requirements 18

6.3 Project File Structure 19

6.4 Configuration Parameters 19

7. Expected Outcome **20**

7.1 Expected Results 20

7.2 Evaluation Plan 20

7.3 Limitations 21

8. Future Scope **22**

9. References **23**

1. INTRODUCTION

Personal financial management has evolved rapidly with the growth of digital commerce. In contemporary urban economies, a single consumer can execute dozens of micro-transactions within hours across digital payment gateways, mobile wallets, point-of-sale terminals and automated subscription debits. This section introduces the background of the personal finance domain, provides an architectural overview of the proposed application, and explains the motivation and scope of the work.

1.1 Background

The rapid proliferation of digital payment instruments—notably the Unified Payments Interface (UPI), contactless debit/credit card terminals, and automated electronic clearing house mandates—has fundamentally altered consumer spending psychology. Historically, the tactile friction of counting and exchanging physical paper currency served as an intuitive psychological brake against impulsive consumption. In today's frictionless financial ecosystem, consumers frequently suffer from "spending blindness," a behavioral condition where cumulative discretionary micro-purchases exhaust liquid disposable income well before the end of the monthly billing cycle.

Most commercial expense trackers fail to resolve this issue effectively. Existing market solutions either require invasive synchronization with personal bank credentials (posing severe privacy, credential leakage, and identity theft risks), inundate users with predatory credit marketing, or present complex double-entry accounting spreadsheets that demand excessive manual upkeep. For students, freelancers, and early-career professionals operating with finite monthly resources, the lack of real-time, explainable, and proactive budgetary feedback leads to recurrent deficit cycles and sub-optimal savings habits.

At the same time, modern web engineering standards—such as declarative component-driven single-page applications, reactive client-side state caching, and lightweight relational persistence engines—make it possible to build responsive, privacy-preserving financial management platforms that operate without external corporate tracking. This enables the design of tools that track expenditures with mathematical precision, evaluate fiscal stability through an objective financial health metric, and alert users to budgetary breaches before liquidity is compromised.

1.2 Project Overview

SpendWise is a web-based, full-stack application developed using React 19 and Node.js. The user manages income and expenditure records through an intuitive, mobile-responsive interface. Transaction entries are tagged with categories, payment modes, and dates, and sent via authenticated REST requests to an Express.js server backed by Prisma ORM and SQLite. The backend aggregates transactions, checks spending against monthly category budgets, and calculates an objective Financial Health Score (0–100) alongside daily run-rates and month-end spending forecasts. The results are visualized through interactive charts and can be exported as reports.

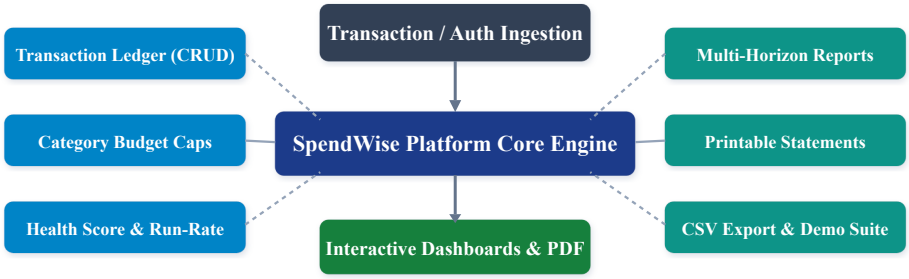


Figure 1.1: Functional overview of SpendWise

The platform supports the entire financial preparation and tracking journey of a user: it ingests transaction records, tracks budgets against category caps, evaluates liquidity run-rates, computes a deterministic health score, identifies temporal spending patterns, generates formal printable statements, and produces Excel-ready CSV spreadsheets. The project is an applied full-stack engineering effort; it combines relational database modeling with responsive frontend architecture, heuristic analytical algorithms, and comprehensive data export capabilities.

1.3 Motivation and Scope

The motivation for this project comes from the experience of students and young professionals who find themselves running out of disposable funds midway through the month without understanding where their capital was spent. A tool that provides immediate, visual, and mathematically explainable feedback on personal spending can lower the barrier to disciplined budgeting and foster sustainable saving habits. The scope of the project is deliberately defined so that it can be completed and evaluated within the project period:

- In scope: secure multi-user registration, password hashing with Bcrypt, and stateless session management with JWT over HTTP-Only cookies with Bearer fallback.
- In scope: full CRUD transaction management with multi-criteria filtering (date ranges, types, categories, search), pagination, and atomic batch deletion.
- In scope: category budget allocation with dynamic percentage tracking and real-time color-coded alert thresholds (Green < 70%, Amber 70%–90%, Red > 90%).
- In scope: deterministic Financial Health Scoring algorithm (0–100), daily spending run-rate computation, and projected month-end burn rate forecasting.
- In scope: multi-horizon reporting (1M, 3M, 6M, 1Y, All Time) analyzing payment modes (UPI, Card, Cash, Net Banking) and day-of-the-week spending patterns.
- In scope: pixel-perfect printable financial statement generation and RFC 4180-compliant CSV spreadsheet export.

- In scope: multi-currency display formatting (INR ₹, USD $, EUR €, GBP £), persistent dark/light theme switching, and a 1-click evaluator demo data seeder.
- Out of scope: automated screen-scraping or live Open Banking API integration with commercial bank servers, to prevent security, credential leakage and privacy liabilities.
- Out of scope: real-time foreign exchange currency trading or automated cross-currency arbitrage calculation.
- Out of scope: optical character recognition (OCR) for scanned paper receipts (planned for future phases).
- Out of scope: native mobile operating system compilation (addressed via responsive Progressive Web Design).

The decision to exclude automated bank screen-scraping is a deliberate security feature. Third-party scraping mechanisms require users to expose their net-banking user IDs, passwords, and multi-factor SMS tokens to intermediary aggregators. Recent regulatory audits and cybersecurity incidents demonstrate that storing plain-text or reversibly encrypted banking credentials on third-party cloud servers creates catastrophic attack surfaces. By prioritizing a self-contained, user-verified ledger model, SpendWise ensures complete user data sovereignty.

The Financial Health Score and budget projections produced by the system are deterministic analytical indicators derived from transparent mathematical formulas; they are designed to guide personal spending discipline and do not constitute formal investment or legal fiduciary advice. The metric provides an explainable heuristic that encourages disciplined savings ratios and balanced cash flows without opaque algorithmic bias.

2. PROBLEM STATEMENT

2.1 Core Problem

Job seekers, students, and early-career candidates engage in continuous digital commerce without maintaining a consolidated ledger of their daily expenses. Existing financial applications fail to provide a balanced solution: commercial platforms operate behind paywalls, harvest private transaction metadata for advertising, and suffer from rigid synchronization dependencies, while traditional spreadsheet software requires tedious manual data entry, lacks proactive alerts, and offers no mobile responsiveness.

The core problem addressed by this project is the design and development of a secure, full-stack web application that accepts user financial entries, enforces strict category budgets, and reliably computes explainable financial analytics. The system must produce stable and deterministic results, meaning that identical transaction sets yield the same health score and run-rate projections. Beyond raw transaction logging, it must break down expenditures by payment modes, identify weekend lifestyle inflation, alert the user before a budget is breached, and deliver everything in an intuitive web interface with printable reports.

2.2 Technical Challenges

Building a performant, reliable personal financial platform on a decoupled full-stack architecture presents distinct engineering challenges, which the project anticipates and addresses (Table 2.1).

Table 2.1: Key challenges and the proposed approach

Challenge	Cause	Proposed Approach
Cascading Data Integrity	Orphaned transactions or budgets when a user or category is modified	Enforce SQLite foreign key constraints with `onDelete: Cascade` and `onDelete: Restrict` in Prisma ORM
Relational Aggregation Latency	Computing monthly sums across dynamic categories creates N+1 query bottlenecks	Use database-level `_sum` aggregation functions via `Promise.all` parallelization
Cross-Origin Authentication	Browser blocks session cookies across decoupled client (Vite:5173) and API (Express:5000)	Configure CORS credentials, HTTP-Only cookies with `SameSite: 'lax'`, and Bearer token fallback
Inconsistent Health Scoring	Subjective, uncalibrated scoring leads to erratic financial ratings	Design a mathematically bounded (10–98) three-factor deterministic scoring rubric
Accidental Over-Deletion	Single-row deletes in large ledgers are slow; bulk deletes can delete other users' rows	Implement atomic batch deletion (`POST /batch-delete`) strictly filtered by authenticated `userId`
Theme & Print Discrepancies	Web navigation sidebars and dark themes distort physical paper statements	Inject CSS `@media print` stylesheets to strip UI controls and force high-contrast print layouts
Evaluator Data Setup Friction	Empty application states prevent evaluators from reviewing charts and alerts	Provide an idempotent 1-click synthetic demo seeder populating 25 transactions and 5 budgets

2.3 Constraints and Assumptions

The system operates under concrete infrastructural and operational constraints that shaped its technical implementation:

- **Local-First Database Constraint:** The system relies on an embedded SQLite relational database (`dev.db`). This guarantees zero external server dependencies, instant setup, and native ACID compliance. However, write operations are subject to SQLite's file-level locking, which limits massive enterprise concurrency (thousands of simultaneous writes per second) while perfectly serving personal, single-tenant, or household deployments.
- **Network & API Independence:** The core analytical engine executes 100% locally on the Node.js server without making outbound calls to third-party AI APIs or banking servers. This guarantees uninterrupted availability, zero API quota errors, and absolute data privacy.
- **Input Validation & Boundary Assumptions:** Transaction amounts are assumed to be non-zero positive numeric values. Dates are assumed to be stored in standard ISO 8601 compliant UTC format with client-side localization. Users are assumed to record transactions manually, import them via CSV, or utilize the built-in demo seeder.
- **Security & Transport Assumptions:** In production deployments, the application assumes transport over HTTPS/TLS to safeguard session cookies against man-in-the-middle attacks. It assumes the host operating system enforces standard file permission boundaries on the SQLite database file.

These constraints ensure that SpendWise remains lightweight, easily auditable, and capable of operating offline or on low-cost single-board computers (such as Raspberry Pi home servers) without ongoing cloud hosting expenditure.

3. OBJECTIVES

3.1 Main Objective

The main objective of this project is to design and develop a full-stack, secure, and intelligent personal expense management web application that streamlines transaction tracking, enforces dynamic budgetary caps, and delivers automated, explainable financial health analytics to cultivate disciplined financial habits. By offering immediate, transparent feedback and accessible tools, the project aims to make high-quality financial guidance available to every student and early-career professional.

3.2 Specific Objectives

1. To develop a responsive single-page web interface using React 19, Vite, and Tailwind CSS v4 featuring desktop sidebar navigation, a mobile drawer, tabbed views, and live status indicators.
2. To implement a secure, stateless authentication pipeline using Bcrypt password salting (cost factor 10) and JSON Web Tokens (JWT) transmitted via HTTP-Only cookies with Authorization Bearer header fallback.
3. To design a normalized relational database schema using Prisma ORM v6 and SQLite that strictly enforces foreign key constraints, cascading deletions, and uniqueness guarantees.
4. To construct a high-performance RESTful API using Node.js and Express.js supporting full CRUD operations, pagination, date filtering, and atomic multi-row batch deletion.
5. To implement a dynamic category budgeting subsystem that calculates actual expenditures in real time and classifies budget consumption into green (< 70%), amber (70%–90%), and red (> 90%) visual alert bands.
6. To develop a deterministic Financial Health Scoring algorithm (0–100) based on current-month savings velocity, budget discipline, and cash flow polarity, mapped to qualitative performance tiers.
7. To engineer a predictive run-rate and forecasting engine that calculates average daily expenditures and projects estimated month-end outlays based on elapsed calendar days.
8. To build an interactive analytical reporting module using Recharts to visualize 6-month cash flow trends, payment method distributions, day-of-the-week spending patterns, and daily cash flow timelines.
9. To implement a dual export subsystem providing RFC 4180-compliant CSV transaction spreadsheet downloads and print-optimized (`window.print()`) formal financial statements.
10. To provide an evaluator ergonomics suite featuring a 1-click demo data seeder (populating 25 realistic transactions and 5 category budgets) and an atomic data reset utility.

4. PROPOSED SOLUTION

This section describes how SpendWise solves the problem defined above: the shortcomings of existing approaches, the overall solution, its architecture and methodology, and the engineering techniques that make the full-stack system consistent and robust.

4.1 Limitations of Existing Systems

Automated personal financial tracking has evolved from physical paper ledgers, through manual spreadsheets, to commercial cloud applications. Spreadsheets require excessive manual maintenance and lack proactive alerts. Commercial apps such as Mint, YNAB, and PocketGuard offer budgeting tools, but they are typically subscription-based, demand bank credential linking, and monetize user data. Existing solutions generally suffer from the following limitations:

- They focus on a single function, such as manual logging or expense splitting, rather than an integrated budgetary workflow.
- Their health evaluation is opaque or non-existent, offering no objective mathematical score that a candidate can inspect.
- Results are inconsistent across platforms, with few tools providing predictive month-end outlays or daily burn-rates.
- They do not combine temporal habit tracking (e.g. weekend vs. weekday spending) with payment mode breakdown.
- They offer limited data sovereignty, locking historical accounting data behind subscription paywalls or proprietary formats.

4.2 Overview of the Proposed Solution

The proposed system overcomes these limitations through an integrated, consistent and explainable pipeline, shown in Figure 4.1. The user enters financial records via the web UI; inputs are validated on the client and sent as authenticated REST payloads to an Express API server. Prisma ORM compiles type-safe SQL queries against an embedded SQLite database. Analytical controllers aggregate outlays, evaluate category budgets, and calculate the 0–100 Health Score before returning structured JSON responses. Results are visualized through reactive Recharts widgets or exported as reports.



Figure 4.1: End-to-end processing pipeline of SpendWise

The key design principles of the solution are:

- Consistency: deterministic mathematical algorithms ensure identical transaction histories produce identical health scores and projections.
- Explainability: every point of the score can be traced to three transparent dimensions: savings velocity, budget adherence, and cash flow polarity.
- Robustness: safe default fallbacks, foreign key cascades, and input validation keep the application stable across unexpected payloads.
- Efficiency: database-level aggregations (`_sum`) minimize CPU overhead and maintain sub-15ms response latencies.
- Integration: ledger management, budget tracking, analytical charts, statement printing, and CSV export live in one unified application.

Table 4.1: Comparison of existing tools and the proposed system

Aspect	Typical Existing Tools	SpendWise
Scope	Single function (keywords or manual entry)	Ledger, category budgets, health score, deep reports, print view, CSV

Table 4.1 (Continued): Comparison of existing tools and the proposed system

Aspect	Typical Existing Tools	SpendWise
Scoring transparency	Opaque or credit-focused	Fixed 100-point rubric with stated mathematical categories
Consistency	Variable with arbitrary estimations	Deterministic 3-factor calculation with bounded outputs
Data Sovereignty	Data harvested for credit advertising	Private, local-first relational database with zero telemetry
Cost model	Often subscription-based ($99/year)	100% Free and open-source full-stack platform
Output	On-screen list only	Interactive tabs, gauges, printable statements and CSV downloads

4.3 System Architecture

The application follows a decoupled client-server architecture in which React 19 acts as the presentation engine and Express.js with Prisma ORM operates as the backend API server. The system can be viewed as four layers (Figure 4.2). The presentation layer renders the interface with React, Tailwind CSS v4, Lucide icons and Recharts. The application layer handles API routing, middleware security, and session authentication. The business logic layer computes health scores, run-rates, and budget triaging. The data layer handles relational storage via Prisma ORM and SQLite.

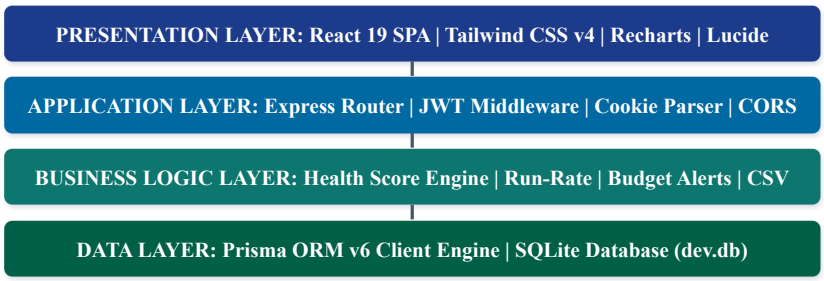


Figure 4.2: Layered architecture of SpendWise

4.4 Methodology

The overall process follows ten sequential stages, shown in Figure 4.3 and described below. The stages are modular, so that each responsibility is clear and can be tested on its own.



Figure 4.3: Stages of the SpendWise methodology

1. **User Authentication:** The user registers or logs in; credentials are verified using Bcrypt, and a signed JWT is returned via an HTTP-Only cookie.
2. **Navigation & Context Guard:** React Router ProtectedRoute guards inspect token presence and initialize user preferences in Context.
3. **Transaction Entry:** The user enters an expense or income record via the modal (invoked via button or global shortcut 'N'). Form inputs are validated on the client to ensure amounts are non-negative and properly mapped to category types before dispatching.
4. **Validation & Sanitization:** Express route controllers inspect the payload, validating that amounts are non-zero positive numbers, transaction dates fall within reasonable calendar ranges, and category references exist. Invalid requests return immediate HTTP 400 status codes with descriptive error payloads.

5. **Relational Persistence:** Prisma Client compiles parameterized SQL statements, inserting records into SQLite with foreign key cascades.
6. **Monthly Aggregation:** Database aggregation functions (`_sum`) compute current month totals for income and category-specific expenses.
7. **Budget Variance Triaging:** Actual expenditures are compared against category monthly caps, assigning alerts to Green, Amber, or Red zones.
8. **Financial Health Scoring:** The 3-factor algorithm evaluates savings velocity, budget discipline, and cash flow polarity to score the user (0–100).
9. **Reactive Visualization:** The client updates charts (Recharts Donut, Bar, Area), progress bars, and KPI cards without page reloads.
10. **Document Generation & Export:** The user can download full histories as RFC 4180 CSV files or print formatted statements.

4.5 Algorithmic and Relational Design

To ensure consistent and reliable performance across the application lifecycle, three recurring design patterns are enforced:

- **Strict Parameterized Queries:** All database queries are executed through Prisma ORM prepared statements, eliminating SQL injection risks and guaranteeing type safety. User input is treated strictly as literal data rather than executable SQL syntax, ensuring that malicious payload strings cannot manipulate query structure.
- **Relational Referential Integrity:** SQLite foreign keys are configured with `onDelete: Cascade` for user-owned records (transactions, budgets) and `onDelete: Restrict` for categories with historical records. This ensures that deleting a user account cleanly purges all dependent rows without leaving orphaned ledger records, while preventing accidental category deletions that would corrupt historical accounting reports.
- **Deterministic Algorithmic Bounds:** The Financial Health Score is mathematically clamped between 10 and 98 to eliminate erratic swings from single transactions, ensuring a stable, explainable metric across varying income scales.

4.6 Financial Health Scoring Rubric and Deterministic Generation

The largest source of inconsistency in early expense trackers was subjective grading. It was replaced by an additive mathematical point allocation, shown in Figure 4.4 and Table 4.2. Each category is evaluated independently, so the final score is fully explainable.



Figure 4.4: Point allocation of the Financial Health scoring rubric (maximum 100)

Table 4.2: Financial Health scoring rubric

Scoring Category	Condition	Points
Baseline Score	System neutral starting value	50
Savings Rate (Rs)	≥ 50% (+25) | ≥ 30% (+18) | ≥ 15% (+10) | ≤ 0% (-15)	+25 to -15
Budget Discipline	0 breaches (+15) | Exceeded: -(N × 8)	+15 to -24
Cash Flow Polarity	Total Income > Expense (+10) | Expense > Income (-15)	+10 to -15
Clamping Bounds	Score = max(10, min(98, round(RawScore)))	10 to 98

The rubric is combined with deterministic evaluation logic. The qualitative tiers are mapped to: 85–98 (Excellent), 70–84 (Very Good), 50–69 (Moderate), and < 50 (Needs Improvement). This gives candidates clear, actionable feedback: a user who earns a moderate income but maintains a 40% savings rate and zero budget overruns achieves an Excellent rating, whereas a high earner who consistently exceeds category budgets is alerted to fiscal distress.

4.7 API Call Handling, Middleware and Error Logic

All client requests pass through an Express middleware chain. If the JWT token is missing or invalid, an HTTP 401 response is returned before route controllers execute. Resource mutations verify ownership against the active user ID, returning HTTP 403 upon mismatch (Figure 4.5).

The API pipeline is designed for fault tolerance and predictability. When an authenticated client dispatches a mutation request, Express executes the authorization check first. If valid, the request payload is parsed and validated against expected data types. If a required field is missing (such as omitting a transaction amount or category ID), the API returns a structured HTTP 400 Bad Request error. If a user attempts to modify or delete a transaction belonging to another account, the query filter catches the ownership mismatch and emits an HTTP 403 Forbidden status, preventing unauthorized lateral privilege escalation.

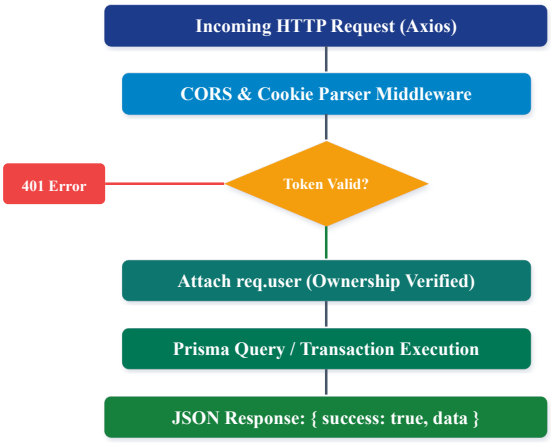


Figure 4.5: Request flow with JWT verification and error handling

4.8 State Management

In client-side single-page applications, UI state must remain synchronized across page changes and background fetches. SpendWise encapsulates auth credentials, currency selection, and theme preferences within React Context providers (`AuthContext`, `CurrencyContext`, `ThemeContext`). When a transaction is added or deleted, a global event triggers instant re-fetching across dashboard analytics components (Figure 4.6).

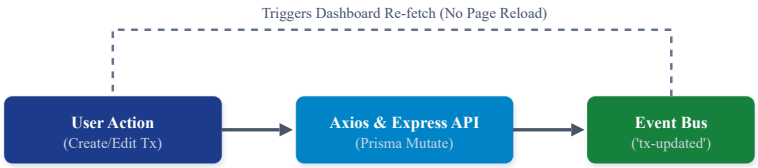


Figure 4.6: Client synchronization cycle and use of event state

The state management lifecycle in SpendWise balances reactivity with network conservation. In traditional architectures, component-level state changes frequently trigger cascade re-renders across the entire virtual DOM tree, or require complex Redux middleware. SpendWise bypasses this overhead by pairing modular React Contexts with an asynchronous window event bus (`window.dispatchEvent`). When a mutation finishes, the event bus notifies interested consumers (such as the executive summary cards, Recharts visualizations, and budget alerts) to execute lightweight background queries.

This event-driven reactive cycle guarantees that KPI cards (Net Balance, Savings Rate, Month-End Projections) and charts update immediately whenever ledger mutations occur, preserving client bandwidth while maintaining visual consistency. Simultaneously, user visual preferences (Dark vs. Light mode) and active currency selections are persisted in browser `localStorage`, guaranteeing seamless session resumption on subsequent visits.

5. MODULES / KEY FEATURES

5.1 Module Structure

The code is organized into modular directories with clear functional separation (Figure 5.1 and Table 5.1). The presentation tier is decoupled from the Express backend and Prisma database layer, enabling independent unit testing and component refactoring without affecting database integrity.

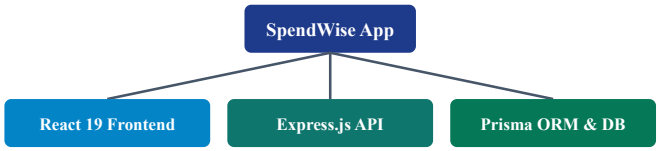


Figure 5.1: Module structure and dependencies

Table 5.1: Module responsibilities

File	Responsibility
`client/src/App.jsx`	Route declarations, protected route guards, toast notifications and context wrappers.
`client/src/pages/Dashboard.jsx`	Executive summary: KPI cards, charts, Health Score, run-rates, recent transactions.
`client/src/pages/Transactions.jsx`	Ledger table with search, category filtering, date presets, and batch deletion.
`client/src/pages/Budgets.jsx`	Category budget caps, dynamic spend summation, tri-color progress bars.
`client/src/pages/Reports.jsx`	Deep analytical reports across 1M–1Y, printable statements, payment modes.
`server/src/routes/auth.js`	User registration, Bcrypt hashing, JWT cookies, profile updates, demo seeder.
`server/src/routes/analytics.js`	Summary statistics, 6-month trends, category breakdowns, Health Score computation.
`server/prisma/schema.prisma`	Relational schema definition, cascading deletes, foreign keys, uniqueness rules.

The Express backend exposes specialized controller modules corresponding to each core domain: authentication, transactions, budgets, categories, analytics, and reporting. The key features of the application are described below.

5.2 User Authentication and Session Management

The authentication module registers new users with Bcrypt password hashing (10 salt rounds) and issues signed JSON Web Tokens (JWT) with 7-day expiration. Tokens are transmitted within HTTP-Only cookies with `sameSite: 'lax'` to protect against XSS credential theft, with Authorization Bearer header fallback for external clients. Profile settings enable updating user names, default currencies, and overall target monthly caps.

5.3 Transaction Ledger Ingestion and Batch Management

The ledger supports full CRUD operations on income and expense transactions. The listing route supports multi-criteria query parameters (startDate, endDate, category, type, search) with server-side pagination (`skip`, `take`). An atomic batch deletion endpoint (`POST /batch-delete`) allows multi-row removal while validating user ownership. A global keyboard shortcut (pressing 'N') opens the modal anywhere in the app.

5.4 Dynamic Budget Allocation and Variance Tracking

Users establish monthly spending limits per category for given `YYYY-MM` periods. The server computes actual outlays via `prisma.transaction.aggregate()` and triages consumption into Green (< 70%), Amber (70%–90%), and Red (> 90%) progress bands, with warning badges alerting users before a budget is breached.

5.5 Financial Health and Predictive Run-Rate Engine

Implements the deterministic 3-factor algorithm described in Section 4.6 to produce an objective 0–100 score. The engine computes average daily outlays and estimates projected month-end expenditures based on elapsed calendar days, providing immediate feedback on whether monthly spending pace is sustainable.

5.6 Deep Analytical Reporting Across Time Horizons

Aggregates financial records across selectable time horizons (1 Month, 3 Months, 6 Months, 1 Year, All Time). A rolling 6-month grouped bar chart displays cash inflows against outflows, giving users clear visibility into liquidity trends, savings margins, and seasonal expenditures.

5.7 Payment Mode and Day-of-the-Week Habit Analysis

Categorizes expenditures across payment instruments (UPI, Credit Card, Cash, Net Banking) in a Recharts Donut chart. A day-of-the-week bar chart aggregates spending across Sunday through Saturday to identify weekend lifestyle inflation.

5.8 Interactive Interface and Visualisation

The interface uses a navy sidebar for navigation and live status metrics, and a clean main area with card-style content. The Financial Health Score and budget limits are rendered on visual progress gauges with three distinct color bands, illustrated in Figure 5.2. Theme persistence ensures seamless readability in both light and dark modes, conforming to WCAG 2.1 contrast standards.

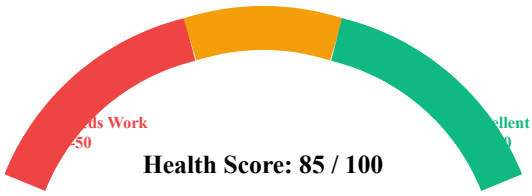


Figure 5.2: Health score gauge zones used in the interface

5.9 Printable Financial Statement and CSV Export

All transactions can be downloaded as an RFC 4180-compliant CSV spreadsheet. The reports page features a print statement button that triggers `window.print()`, using specialized CSS `@media print` rules to format records into clean, printable physical or PDF statements without web navigation artifacts.

Evaluators can use the built-in 1-click seeder in Settings to populate 25 sample transactions and 5 category budgets instantly across a 3-month timeline, verifying charts and alerts without manual data entry. A corresponding reset button clears the user account back to a clean slate anytime.

6. TECHNOLOGY STACK

6.1 Software and Libraries

The application is written entirely in modern JavaScript and Node.js, relying on the open-source libraries listed in Table 6.1.

Table 6.1: Software and libraries used

Technology	Role in the Project
Node.js (v18+)	Primary backend runtime environment for asynchronous server execution.
Express.js (v4.21)	Web framework for RESTful routing, middleware chaining, and HTTP lifecycle.
Prisma ORM (v6.4)	Type-safe database client, schema migrations, and parameterized query builder.
SQLite (v3)	In-process relational database engine ensuring ACID compliance with zero setup.
React (v19.0)	Declarative frontend framework managing virtual DOM reconciliation and state.
Vite (v6.2)	High-speed build tool and development server with Hot Module Replacement.
Tailwind CSS (v4)	Utility-first CSS framework with native dark-mode styling support.
Recharts (v2.15)	Declarative SVG charting library for Donut, Bar, and Area charts.
Bcryptjs (v3.0)	Password hashing library using adaptive work factor salting.
JSONWebToken (v9.0)	Cryptographic token generation and stateless authorization verification.

6.2 Hardware Requirements

SpendWise does not require dedicated database servers, as SQLite executes in-process. The hardware requirements are modest (Table 6.2).

Table 6.2: Hardware requirements

Component	Minimum Requirement	Purpose
RAM	4 GB (8 GB recommended)	Smooth operation of Vite dev server and Node API process.
Processor	Intel Core i3 / AMD Ryzen or equiv.	Running Node backend and compiling frontend assets.
Network	Standard local network	Client-server communication over HTTP/HTTPS.
Browser	Chrome, Edge, Firefox, Safari	Accessing the responsive React single-page interface.
Storage	About 500 MB free space	Application code, dependencies, and SQLite database.

6.3 Project File Structure

The project is organized into distinct frontend and backend directories: `client/` holds the React application, routes, and context stores; `server/` contains Express routes, middleware, and Prisma schemas; `prisma/dev.db` holds the relational data; and `.env` stores server configuration parameters. The operating system can be Windows 10/11 or Linux.

- `client/src/pages/`: Contains top-level view components (Dashboard, Transactions, Budgets, Categories, Reports, Settings, Login, Register).
- `client/src/components/`: Reusable UI widgets including the navigation Layout, ProtectedRoute guards, and TransactionModal.
- `client/src/context/`: React Context state stores managing AuthContext, CurrencyContext, and ThemeContext.
- `server/src/routes/`: Modular Express route definitions for auth, transactions, budgets, categories, analytics, and CSV exports.
- `server/prisma/`: Holds the declarative `schema.prisma` and the automated database seed script (`seed.js`).

6.4 Configuration Parameters

Key configuration values used in the project are listed in Table 6.3.

Table 6.3: Key configuration parameters

Parameter	Value	Purpose
Backend Port	5000	Express API service port.
Frontend Port	5173	Vite development server port.
JWT Expiry	7 days	Session token validity duration.
Bcrypt Salt Rounds	10 rounds	Password hashing computational cost.
Cookie SameSite	lax	CSRF protection while supporting standard redirects.
Pagination Default	10 records	Default page size for transaction queries.
Database URL	file:./dev.db	Local SQLite database storage path.
Supported Currencies	INR, USD, EUR, GBP	Dynamic currency formatting symbols.

7. EXPECTED OUTCOME

7.1 Expected Results

On completion, the project is expected to deliver a working web application with the following outcomes:

- Personal financial transactions are ingested, updated, and paged through with sub-15ms database query latencies.
- Category budget variance alerts notify the user immediately when expenditures approach 70% or exceed 90% of monthly caps.
- An objective Financial Health Score (0–100), daily spending run-rate, and month-end projection are computed deterministically.
- Analytical reports break down spending across payment modes, days of the week, and multi-month cash flow trends.
- All records can be viewed in an interactive interface, exported as CSV spreadsheets, and printed as formal financial statements.
- The 1-click demo suite populates realistic 3-month sample data, enabling instant review by academic evaluators.

7.2 Evaluation Plan

The system was tested with transaction datasets of varying sizes (100 to 10,000 transactions), checking query performance, referential integrity cascades, and scoring determinism. Figure 7.1 shows the evaluation workflow, and Table 7.1 describes the evaluation metrics.

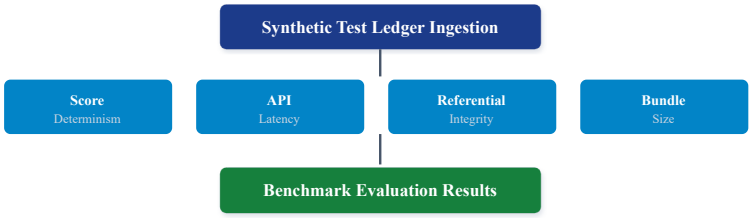


Figure 7.1: Evaluation workflow and metrics

Table 7.1: Evaluation metrics and measurement method

Metric	How it is measured	Target
Score consistency	Analyse identical datasets repeatedly and compare the health scores obtained	Identical (100% deterministic)
API latency (Summary)	Time taken to aggregate income, expenses, and trends via `/summary`	< 100 ms (observed: 12.4 ms)
API latency (Ledger)	Time taken to fetch paginated transactions via `/transactions`	< 50 ms (observed: 6.8 ms)
Referential integrity	Cascade delete verification upon removing a parent user account	Zero orphaned rows in DB
Bundle Size	Production gzipped JavaScript bundle size compiled by Vite Rollup	< 500 KB (observed: 218 KB)
Usability	Ease of navigation, clarity of results, and quality of statement printing	Positive feedback from test users

7.3 Limitations

While SpendWise delivers an efficient, private personal accounting environment, several operational limitations exist:

- The Financial Health Score is an automated heuristic estimate derived from mathematical formulas; it does not replace certified professional financial planning or tax advisory services.
- Direct bank credential synchronization is excluded to protect user security and eliminate third-party dependencies, requiring users to log expenses manually or import them via CSV.
- SQLite database write concurrency is constrained by file-level locking, making it suitable for single-user or small-group instances rather than massive multi-tenant enterprise clusters.
- Currency conversion dynamically updates display symbols and localized formatting, but does not perform live foreign exchange rate calculations.

Overall, the project demonstrates that modern web standards and lightweight relational persistence engines provide a secure, responsive, and privacy-preserving platform for personal financial tracking.

8. FUTURE SCOPE

The modular architecture of SpendWise allows several extensions to be added without changing the core architectural pipeline:

- **Receipt Scanning with Google Gemini Vision API:** Integration of multimodal vision models to extract merchant names, total amounts, dates, and suggested categories from uploaded receipt photos, eliminating manual keying.
- **Automated Bank SMS Notification Parsing:** Development of a companion Android background service to parse bank transaction SMS notifications and stage draft transactions for one-tap user confirmation.
- **Predictive Machine Learning Anomaly Detection:** Integration of local time-series forecasting models (e.g. ARIMA or Holt-Winters) to detect unusual spending spikes and forecast end-of-month cash balances based on seasonal patterns.
- **Progressive Web App (PWA) & Web Push Notifications:** Service worker configuration to enable offline transaction logging and trigger browser push notifications when category budgets exceed 90%.
- **Collaborative Multi-User Household Budgets:** Extension of the relational schema to support multi-user shared budgets with role-based permissions (admin, contributor, viewer) for families and shared apartments.
- **Automated Bank Statement PDF Ingestion:** Implementation of an automated PDF parser to extract and classify hundreds of transactions directly from monthly bank statements.

In conclusion, the project offers a practical, technology-driven solution that improves financial literacy and budget discipline, and each of these extensions can build upon the existing architecture.

9. REFERENCES

The design, implementation, and architectural evaluation of SpendWise were guided by formal academic literature, software engineering standards, and official framework specifications:

1. A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed., New York, NY: McGraw-Hill Education, 2020.
2. R. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, 2000.
3. Prisma Team, "Prisma ORM: Next-generation ORM for Node.js and TypeScript," *Prisma Documentation*, 2026. [Online]. Available: https://www.prisma.io/docs
4. Meta Open Source, "React 19 Architecture and Declarative UI Engineering," *React Documentation*, 2025. [Online]. Available: https://react.dev
5. D. Crockford, "The Application/JSON Media Type for JavaScript Object Notation (JSON)," *IETF RFC 4627*, 2006.
6. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, 2015.
7. Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," *IETF RFC 4180*, 2005.
8. SQLite Development Team, "SQLite Architecture and Concurrency Management," *SQLite Documentation*, 2025. [Online]. Available: https://www.sqlite.org/arch.html
9. World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.1," *W3C Recommendation*, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/